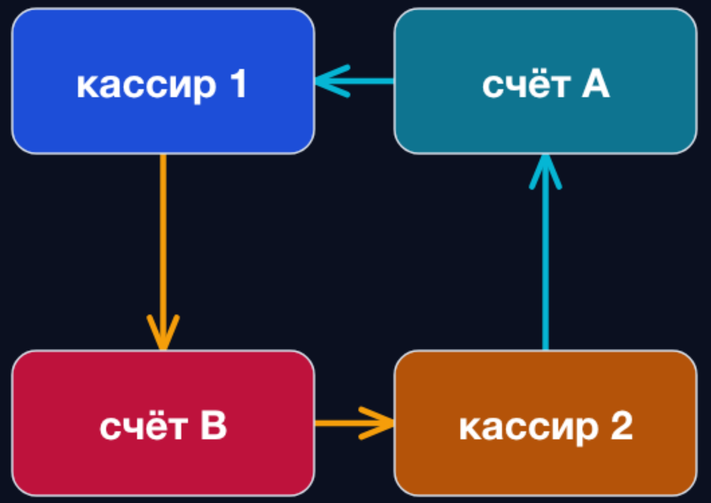
ПРОЕКТ 2

Взаимоблокировки в банковской системе

Вариант 2 · диагностика, предотвращение и обнаружение



Цикл ожидания

Исходная взаимоблокировка

Оба потока остановились на строке with locks[dst]



B принадлежит кассиру 2

A принадлежит кассиру 1

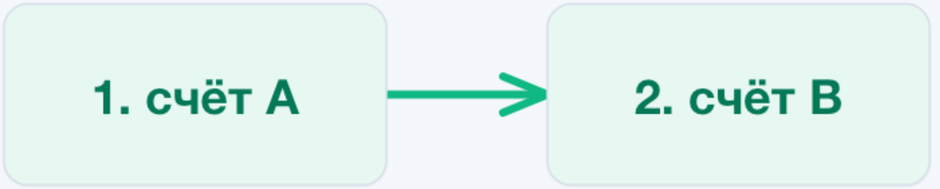
faulthandler: оба рабочих потока на строке 22
главный поток ждёт join()

Условия Коффмана и исправление

Для deadlock нужны все четыре условия

1. Взаимное исключение
2. Удержание и ожидание
3. Нет принудительного изъятия
4. **Циклическое ожидание**

Исправление: единый порядок ресурсов



Оба направления перевода берут блокировки A, затем B

Результат: A + B = 2000 зависания нет

Алгоритм банкира

Need = Max – Allocation

Процесс завершается условно, если Need ≤ Work. После этого Work увеличивается на Allocation процесса.

bank_state.txt
SAFE

Available = [4, 2, 1]

bank_state2.txt
UNSAFE

Available = [4, 1, 1]

Безопасная последовательность



Во втором состоянии после P4 остаются P0, P1, P2 и P3: ни один не проходит проверку Need ≤ Work.

Проверка запросов ресурсов

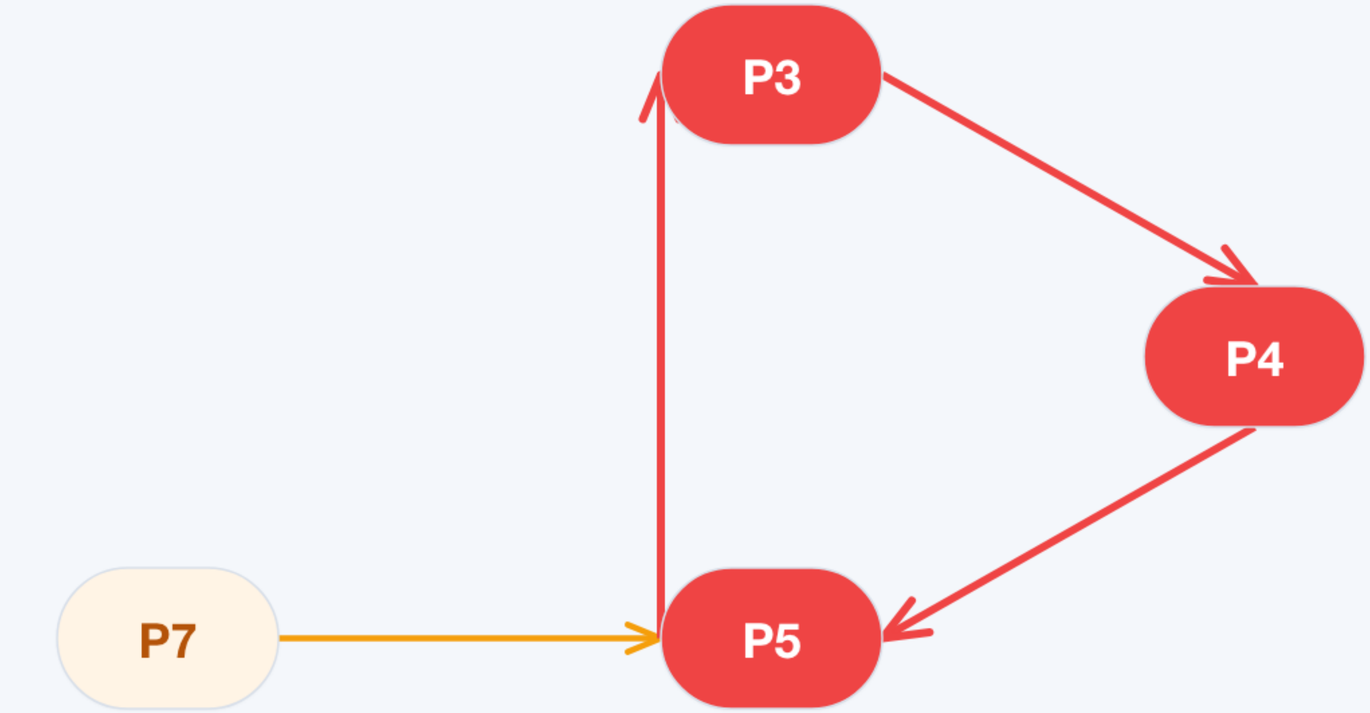
Каждый запрос проверен относительно исходного bank_state.txt

Процесс	Запрос A B C	Решение	Причина
P2	1 0 0	GRANT	состояние остаётся SAFE
P0	0 1 0	REFUSE	состояние становится UNSAFE
P3	0 1 0	GRANT	состояние остаётся SAFE
P4	2 0 0	ERROR	превышен объявленный Need
P1	1 0 1	REFUSE	состояние становится UNSAFE

GRANT: 2 REFUSE: 2 ERROR: 1

Граф ожидания

Ребро: ожидающий процесс → владелец ресурса



P7 зависит от цикла

Цикл: P3, P4, P5



P2 сам не заблокирован

TrackedLock: обнаружение до зависания



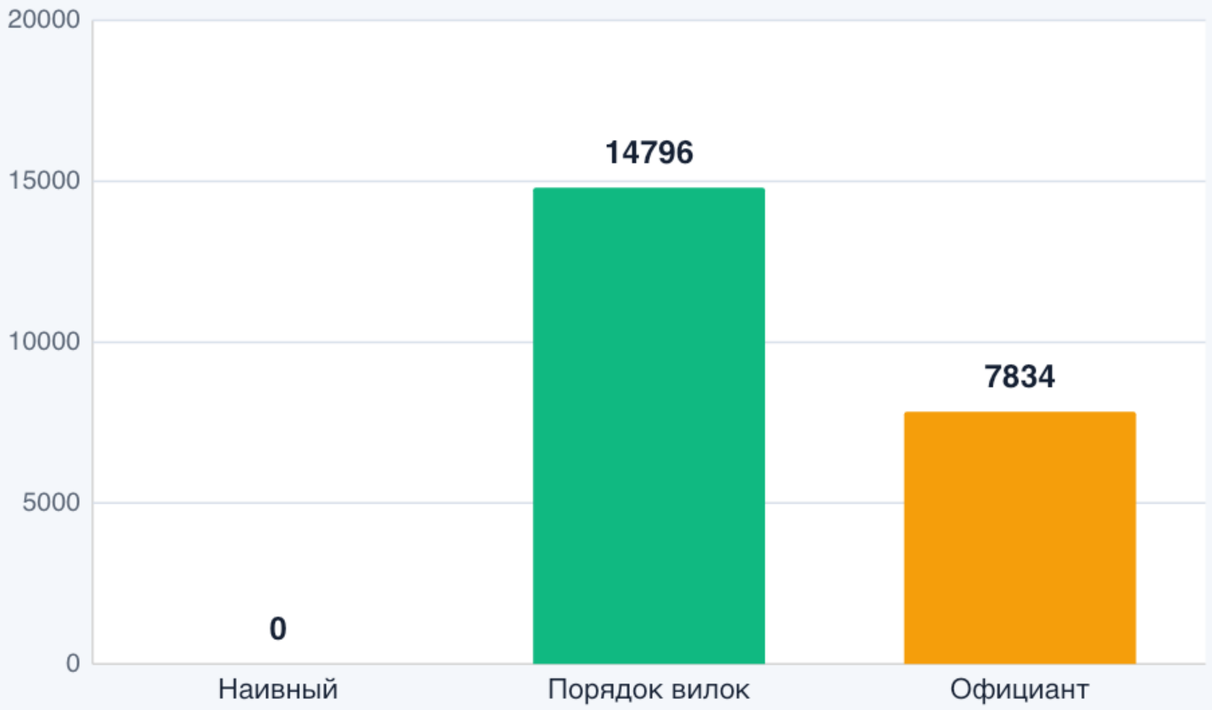
Цикл найден: кассир-1 → кассир-2 → кассир-1

Исключение разворачивает with, освобождает удерживаемый счёт и не оставляет зависших потоков.

Проверка: оба потока завершены · сумма денег = 2000

Обедающие философы: 10 секунд

Число приёмов пищи



Фактический проверочный запуск; числа могут меняться из-за планировщика ОС.

Итог

Наивный вариант
5 зависших потоков

Порядок вилок
deadlock отсутствует

Семафор на 4 места
deadlock отсутствует

Предотвращать · избегать · обнаруживать